Experiment No -04 (Group A)

Title:-

Sorting Employee Salaries using Selection Sort and Bubble Sort in Python

Problem statement:-

In a company, employee salaries are stored in a list as floating-point numbers. Write a Python program that sorts the employee salaries in ascending order using the following two algorithms:

•**Selection Sort:** Sort the salaries using the selection sort algorithm.

•**Bubble Sort:** Sort the salaries using the bubble sort algorithm.

After sorting the salaries, the program should display top five highest salaries in the company

Pre-requisite:-

•Basic understanding of python programming.

•Knowledge of variables, array, loops, and functions.

•Understanding of conditional statements (if, else).

•Basic knowledge of sorting algorithms such as Selection Sort and Bubble Sort.

Objective:-

To write a Python program that:

1. Accepts employee salaries from the user.
2. Sorts the salaries using **Selection Sort** and **Bubble Sort** algorithms.
3. Displays the **top five highest salaries** after sorting.

Software and Hardware requirements:-

1. **Operating System recommended:-** 64-bit Open source Linux or its derivative
2. **Programming tools recommended: -** Python
3. **Programming IDE like:** Python 3 / Jupyter Note Book / Visual Studio

Outcome:-

After completing this experiment, the student will be able to: Implement **Selection Sort** and **Bubble Sort using python**, Displayed salaries in ascending order and the **top five highest salaries** shown after sorting.

Theory:-

Selection Sort Algorithm (Ascending order)

1. Start
2. Read the number of elements n.
3. Read the array arr containing n elements.
4. For i = 0 to n - 1

   1.do Set Min = i.

   2.For j = i + 1 to n - 1 do

   If arr[j] < arr[Min] then

   Set Min = j.

   3.Swap arr[i] and arr[Min] using a temporary variable:

   temp = arr[i]

   arr[i] = arr[Min]

   arr[Min] = temp

5.Display the sorted array.

6.Stop

Consider the following depicted array as an example.

[14, 33, 27, 10, 35, 19]

Number of elements (n) = 6

Pass 1 (i = 0)

- Assume **Min = 0** → 14
- Compare with remaining elements:
  - 33 > 14 → No change
  - 27 > 14 → No change
  - **10 < 14 → Min = 3**
  - 35 > 10 → No change
  - 19 > 10 → No change

Swap arr[0] and arr[3]

Before Swap: [14, 33, 27, 10, 35, 19]

After Swap: [10, 33, 27, 14, 35, 19]

Pass 2 (i = 1)

- Assume **Min = 1** → 33
- Compare:
    - **27 < 33 → Min = 2**
    - **14 < 27 → Min = 3**
    - 35 > 14 → No change
    - 19 > 14 → No change

Swap arr[1] and arr[3]

Before Swap: [10, 33, 27, 14, 35, 19]

After Swap: [10, 14, 27, 33, 35, 19]

Pass 3 (i = 2)

- Assume **Min = 2** → 27
- Compare:
    - 33 > 27 → No change
    - 35 > 27 → No change
    - **19 < 27 → Min = 5**

Swap arr[2] and arr[5]

Before Swap: [10, 14, 27, 33, 35, 19]

After Swap: [10, 14, 19, 33, 35, 27]

Pass 4 (i = 3)

- Assume **Min = 3** → 33
- Compare:
    - 35 > 33 → No change
    - **27 < 33 → Min = 5**

Swap arr[3] and arr[5]

Before Swap: [10, 14, 19, 33, 35, 27]

After Swap: [10, 14, 19, 27, 35, 33]

Pass 5 (i = 4)

- Assume **Min = 4** → 35
- Compare:
    - **33 < 35 → Min = 5**

Swap arr[4] and arr[5]

Before Swap: [10, 14, 19, 27, 35, 33]

After Swap: [10, 14, 19, 27, 33, 35]

Pass 6 (i = 5)

Only one element remains.

No swap required.

Final Array: [10, 14, 19, 27, 33, 35]

Algorithm of bubble Sort

1. **Start**
2. Read the number of elements n.
3. Read the array arr containing n elements.
4. **For** i = 0 **to** n - 2 **do**

    For j = 0 **to** n - 2 **do**

        1. **If** arr[j] > arr[j + 1] **then**

    temp = arr[j]

    arr[j] = arr[j + 1]

    arr[j + 1] = temp

5.Display the sorted array.

6.Stop

Example of Bubble sort algorithm:- [14, 33, 27, 35, 10]

Number of elements (n) = 5

Pass 1 (i = 0)

Step 1: Compare 14 and 33

- 14 < 33 → No Swap

  Array: [14, 33, 27, 35, 10]

Step 2: Compare 33 and 27

- 33 > 27 → Swap

Array: [14, 27, 33, 35, 10]

Step 3: Compare 33 and 35

- 33 < 35 → No Swap

  Array: [14, 27, 33, 35, 10]

Step 4: Compare 35 and 10

- 35 > 10 → Swap

  Array: [14, 27, 33, 10, 35]

Array after Pass 1: [14, 27, 33, 10, 35]

Pass 2 (i = 1)

Step 1: Compare 14 and 27

- 14 < 27 → No Swap

  Array: [14, 27, 33, 10, 35]

Step 2: Compare 27 and 33

- 27 < 33 → No Swap

  Array: [14, 27, 33, 10, 35]

Step 3: Compare 33 and 10

- 33 > 10 → Swap

  Array: [14, 27, 10, 33, 35]

Step 4: Compare 33 and 35

- 33 < 35 → No Swap

  Array: [14, 27, 10, 33, 35]

Array after Pass 2: [14, 27, 10, 33, 35]

Pass 3 (i = 2)

Step 1: Compare 14 and 27

- 14 < 27 → No Swap

  Array: [14, 27, 10, 33, 35]

Step 2: Compare 27 and 10

- 27 > 10 → Swap

  Array: [14, 10, 27, 33, 35]

Step 3: Compare 27 and 33

- 27 < 33 → No Swap

  Array: [14, 10, 27, 33, 35]

Step 4: Compare 33 and 35

- 33 < 35 → No Swap

  Array: [14, 10, 27, 33, 35]

Array after Pass 3: [14, 10, 27, 33, 35]

Pass 4 (i = 3)

Step 1: Compare 14 and 10

- 14 > 10 → Swap

  Array: [10, 14, 27, 33, 35]

Step 2: Compare 14 and 27

- 14 < 27 → No Swap

Step 3: Compare 27 and 33

- 27 < 33 → No Swap

Step 4: Compare 33 and 35

- 33 < 35 → No Swap

Array after Pass 4: [10, 14, 27, 33, 35]

Final Sorted Array [10, 14, 27, 33, 35]

Conclusion:-

- The program successfully implemented **Selection Sort** and **Bubble Sort** on user-entered employee salaries.
- It correctly identified and displayed the **top five highest salaries**.
- This practical helped understand and apply basic sorting algorithms on real-life data.

Viva Questions:-

1. What is sorting?
2. What is Selection Sort?
3. What is Bubble Sort?
4. What is the time complexity of Selection Sort?
5. What is the time complexity of Bubble Sort?
6. What is the space complexity of both algorithms?
7. Which algorithm performs fewer swaps?
8. Which algorithm is stable?
9. How many passes are required for an array of size **n**?
10. Why is Bubble Sort called "Bubble Sort"?
11. Why is Selection Sort called "Selection Sort"?
12. What are the applications of sorting?
13. What is the difference between Selection Sort and Bubble Sort?
14. Which sorting algorithm is better among Selection Sort and Bubble Sort?
15. Which sorting algorithms are preferred for large datasets?